Backend Developer

Complete Learning Notes

6 topics | Foundations, worked examples and practice solutions

Scope: The selected career requirements and all transitive prerequisites in the current CareerIQ catalog.

Use this book in prerequisite order. Start with Foundations if a topic is new, then work through the detailed concepts and examples. Experienced learners can focus on tradeoffs, interview questions and applied practice.

These are original AI-assisted study notes, not an accredited textbook or evidence of mastery. Validate software examples against the linked primary documentation and complete the existing assessments or reviewed projects.

Reference books are listed by title and author only. No third-party textbook PDFs are included.

Contents

1. Python (Advanced) 3

2. Data Structures & Algorithms 10

3. Docker & Containerization 16

4. Advanced SQL & Data Modeling 22

5. REST API & Service Design 28

6. Automated Testing & CI/CD 35

1. Python (Advanced)

Prerequisites: No prerequisites in the current competency catalog.

Python has become the lingua franca for many advanced software tasks, from high-performance networking to low-level system integration. Mastering the advanced features of the language - concurrency with AsyncIO, the power of metaclasses and decorators, and the ability to profile memory or extend Python with C - enables developers to write code that is both expressive and efficient. This chapter introduces these capabilities in a self-contained way, starting from the core ideas and building toward practical implementations. By the end, a beginner will understand when to use asynchronous constructs, how to shape class creation dynamically, and how to measure and improve memory usage, preparing them for real-world projects and technical interviews.

Foundations

AsyncIO is Python's built-in library for cooperative multitasking. Unlike pre-emptive threads, coroutines voluntarily yield control, allowing a single thread to manage many I/O-bound tasks without the overhead of context switches. The event loop schedules these coroutines, and futures represent pending results. Metaclasses are the "class of a class"; they intercept class creation, enabling automatic attribute injection or validation. Decorators are higher-order functions that wrap other callables, adding behavior without modifying the original code. Together they support clean, reusable patterns. Memory profiling in Python involves tools such as tracemalloc and objgraph, which track allocation sites and object graphs. When Python's performance limits are reached, C-extensions let developers write critical sections in C, exposing functions to Python via the CPython API. Understanding reference counting, the Global Interpreter Lock (GIL), and the distinction between pure Python objects and native buffers is essential before venturing into extensions. These foundations form a toolkit for writing scalable, maintainable, and high-performance Python applications.

AsyncIO

AsyncIO provides an event-driven framework where coroutines, defined with async def, can pause at await points. The await keyword yields control back to the loop, which then runs other ready coroutines. This model excels for I/O-heavy workloads such as web servers, database clients, or file processing, because the thread never blocks on network or disk latency. Trade-offs include added complexity in error handling and debugging, as stack traces become fragmented across await boundaries. Moreover, CPU-bound work still blocks the loop unless offloaded to a thread or process pool.

Worked reasoning: Consider a simple HTTP fetcher that retrieves three URLs concurrently. The coroutine fetch(url) opens an aiohttp.ClientSession, awaits response.read(), and returns the length. The main coroutine gathers the three fetch calls with asyncio.gather, allowing the event loop to interleave network waits. The total runtime approximates the longest single request, not the sum, demonstrating true concurrency without threads.

Metaclasses & Decorators

Metaclasses intervene during class creation by overriding __new__ or __init__ in a subclass of type. They can inject methods, enforce naming conventions, or register classes in a registry automatically. Decorators, on the other hand, are applied to functions or methods at definition time, returning a wrapped callable that can log, cache, or modify arguments. Using both together yields powerful patterns: a metaclass can apply a decorator to every method of a class, ensuring uniform behavior such as timing or access control. The main trade-off is readability; excessive metaprogramming can obscure intent, making maintenance harder for newcomers.

Worked reasoning: A metaclass AutoTimer scans a class dictionary, wrapping each callable with a timing_decorator that records start and end timestamps. When an instance method is called, the decorator prints the elapsed time. Defining class MyWorker(metaclass=AutoTimer) automatically equips all its methods with timing without manually decorating each one.

Memory Profiling & C-Extensions

Memory profiling starts with tracemalloc.start() to capture allocation snapshots. By comparing snapshots, developers can pinpoint functions that allocate the most bytes. objgraph.show_refs can visualize reference cycles that prevent garbage collection. When profiling reveals hot spots, a C-extension can replace a Python loop with a compiled routine, reducing per-iteration overhead and memory churn. The CPython API requires careful reference counting: Py_INCREF and Py_DECREF manage object lifetimes, and the GIL must be held when manipulating Python objects. The trade-off is development effort and portability; C code must be compiled for each target platform, and bugs can crash the interpreter.

Worked reasoning: A pure-Python function sum_range(n) computes the sum of squares using a for loop, allocating a new integer each iteration. Profiling shows O(n) allocations. Rewriting the core loop in C as sum_range_c(PyObject *self, PyObject *args) uses a single PyLong_FromLong result, dramatically lowering allocation count and memory usage. The extension is built with setuptools' Extension module and imported as import sum_ext.

Worked examples

Worked example

```
import asyncio
import aiohttp

async def fetch(url):
    async with aiohttp.ClientSession() as s:
        async with s.get(url) as r:
            data = await r.read()
            return len(data)

async def main():
    urls = ['https://example.com', 'https://httpbin.org/ip',
    'https://httpbin.org/uuid']
    lengths = await asyncio.gather(*(fetch(u) for u in urls))
    print('Lengths:', lengths)

asyncio.run(main())
```

The script defines fetch as an async coroutine that opens a client session, performs a GET request, and awaits the response body. main builds a list of three URLs and uses asyncio.gather to schedule all fetch

calls concurrently. Because each await yields while the network is idle, the total runtime is close to the slowest request rather than the sum of three round-trips. Limitations include the need for an event loop (cannot be called from a blocking context without run_until_complete) and that CPU-bound work inside fetch would still block the loop.

Applied example

```
# Pseudocode for a metaclass that decorates all methods
class AutoLog(type):
    def __new__(mcls, name, bases, dct):
        def log_wrapper(func):
            def inner(*args, **kw):
                print(f'Calling {func.__name__}')
                return func(*args, **kw)
            return inner
        for attr, val in dct.items():
            if callable(val):
                dct[attr] = log_wrapper(val)
        return super().__new__(mcls, name, bases, dct)

class Service(metaclass=AutoLog):
    def start(self):
        print('started')
    def stop(self):
        print('stopped')

svc = Service()
svc.start()
svc.stop()
```

AutoLog is a metaclass that iterates over the class dictionary during creation. For each callable attribute, it replaces the original function with a wrapped version that prints a log line before invoking the original. When Service is defined, both start and stop are automatically logged without explicit @decorator usage. This pattern is useful for cross-cutting concerns like logging, authentication, or timing. The trade-off is that every method, including private helpers, receives the wrapper, which may add overhead or interfere with introspection tools.

Process flow

1. 1. Identify a performance or architectural problem (e.g., many concurrent I/O calls).
2. 2. Choose AsyncIO if the workload is I/O-bound; rewrite blocking functions as async coroutines.
3. 3. Insert await points at each I/O operation and run coroutines via asyncio.run or an existing loop.
4. 4. If repetitive cross-cutting behavior is needed, design a decorator or metaclass to apply it uniformly.
5. 5. Profile memory with tracemalloc.start(); capture snapshots before and after the critical section.
6. 6. Analyze allocation differences; if Python overhead is high, implement the hot loop as a C-extension.
7. 7. Test the integrated solution, verify correctness, and measure speed and memory improvements.

Interview and exam checkpoints

Explain how AsyncIO differs from threading and when you would prefer one over the other.

AsyncIO uses a single thread with an event loop that schedules coroutines, allowing many I/O operations to progress without OS thread switches. Threading creates multiple OS threads, each with its own stack and context, incurring higher memory usage and synchronization costs. Prefer AsyncIO for high-concurrency I/O-bound tasks like web servers, where the GIL is not a bottleneck. Use threading when you need true parallelism for CPU-bound work or when interacting with libraries that block the interpreter and cannot be made async.

What is a metaclass in Python and how does it differ from a class decorator?

A metaclass is a subclass of type that controls the creation of classes; its __new__ or __init__ runs when the class statement is executed, allowing injection or modification of attributes before the class object exists. A class decorator is a function that receives a class object after it has been created and returns a (possibly modified) class. Metaclasses operate at class definition time, affecting the class's structure, while decorators act on the finished class, typically adding behavior. Metaclasses are more powerful but also more complex and less commonly needed.

How does tracemalloc help you find memory leaks, and what are its limitations?

tracemalloc records the call stack of each memory allocation, enabling you to take snapshots and compare them to see which lines allocate the most bytes. By filtering snapshots you can locate unexpected growth in specific functions, indicating a leak. Limitations include that it only tracks Python-level allocations, not memory allocated by native extensions, and it adds overhead (typically 5-10%). It also cannot detect reference cycles that keep objects alive unless you inspect the object graph with additional tools.

Describe the steps to safely expose a C function to Python using the CPython API.

First, write a C function that follows the PyCFunction signature, accepting PyObject *self and PyObject *args. Parse arguments with PyArg_ParseTuple, perform the computation, and return a new reference using PyLong_FromLong or similar. Next, create a method table (PyMethodDef array) listing the function name, pointer, flags, and docstring. Define a module initialization function (PyMODINIT_FUNC PyInit_modname) that calls PyModule_Create with a PyModuleDef struct containing the method table. Finally, compile the source into a shared library using setuptools' Extension module, and import it in Python. Throughout, manage reference counts with Py_INCREF/Py_DECREF and hold the GIL when interacting with Python objects.

Summary

Advanced Python equips developers with tools to handle massive concurrency, dynamic class behavior, and low-level performance tuning. AsyncIO replaces threads for I/O-bound workloads by using cooperative coroutines and an event loop, yielding efficient scaling with minimal overhead. Metaclasses and decorators enable systematic injection of cross-cutting concerns, allowing clean, reusable modifications to classes and functions. Memory profiling with tracemalloc and objgraph reveals hidden allocation patterns, guiding decisions to rewrite hot paths as C-extensions when Python's interpreter overhead becomes a bottleneck. Together, these techniques form a cohesive strategy: identify bottlenecks, apply the appropriate abstraction, measure impact, and iterate. Mastery of these concepts prepares engineers for real-world systems where responsiveness, resource usage, and maintainability are paramount.

Practice questions

1. Write an async function that reads three files concurrently and returns their total line count. Use only standard-library asyncio and the third-party aiofiles package.

2. Create a metaclass that automatically registers every subclass in a global registry dictionary keyed by class name.

3. Using tracemalloc, compare memory usage of a list comprehension versus a generator expression that produces the squares of numbers 1..1000. Report the difference in bytes.

4. Design a small C-extension that provides a function fast_sum(arr) returning the sum of a Python list of integers. Outline the steps and explain reference-count handling.

5. Explain a scenario where using a decorator for caching (memoization) could degrade performance instead of improving it.

6. You have a web service that must handle 10,000 simultaneous connections, each performing short HTTP GETs. Choose between AsyncIO and a thread pool, justify your choice, and sketch the high-level code structure.

Practice solutions

1. import asyncio, aiofiles

```
async def count_lines(path):
    async with aiofiles.open(path, 'r') as f:
        cnt = 0
        async for _ in f:
            cnt += 1
        return cnt

async def total():
    files = ['a.txt', 'b.txt', 'c.txt']
    results = await asyncio.gather(*(count_lines(p) for p in files))
    return sum(results)

print(asyncio.run(total()))
# aiofiles delegates ordinary file operations to worker threads, counts lines asynchronously, and aggregates the counts. Because file I/O yields control, the three reads overlap, potentially overlapping
```

waits; speedup depends on the filesystem and workload.

2. registry = {}

```
class RegisterMeta(type):
    def __new__(mcls, name, bases, dct):
        cls = super().__new__(mcls, name, bases, dct)
        registry[name] = cls
        return cls

class Base(metaclass=RegisterMeta):
    pass

class Foo(Base):
    pass

class Bar(Base):
    pass

print(registry) # {'Base': <class '__main__.Base'>, 'Foo': <class '__main__.Foo'>, 'Bar': <class
'__main__.Bar'>}
# Each time a new subclass is defined, RegisterMeta inserts it into the registry, enabling factory patterns or
plugin discovery without manual bookkeeping.
```

3. import gc, tracemalloc

```
def measure(factory):
    gc.collect()
    tracemalloc.start()
    value = factory() # retain the object while measuring
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return current, peak

list_bytes = measure(lambda: [i*i for i in range(1, 1001)])
generator_bytes = measure(lambda: (i*i for i in range(1, 1001)))
print("list current/peak:", list_bytes)
print("generator current/peak:", generator_bytes)
print("current difference:", list_bytes[0] - generator_bytes[0])
```

Results depend on the Python build and allocator; do not hardcode a byte count. The list retains computed elements. The generator retains its suspended state and produces values when consumed. This measures construction, not the work or memory of consuming and retaining all generated results. Discarding both objects before snapshots would not measure their retained memory.

4. Define a method table and module initialization function, then expose a callable that parses an iterable argument. Obtain an iterator with PyObject_GetIter and release its new reference on every exit path. Each non-null PyIter_Next result is also a new reference: process it, then decrement it. A null iteration result can mean exhaustion or an exception; check PyErr_Occurred after the loop. For arbitrary Python integers, maintain a Python integer accumulator using checked PyNumber_Add results and correct reference ownership. A C long accumulator can overflow even when each input fits individually; do not use unchecked signed addition. Return a new reference to the result on success and NULL with an exception

set on failure. Include Python.h, define the method table/module entry point and use an appropriate extension build configuration. Test empty input, non-integers, huge integers, an iterator that raises, and repeated calls under leak checking. A C extension is not automatically faster; benchmark against Python's built-in sum before accepting its maintenance cost.

5. If the decorated function is called with a large number of unique arguments, the cache grows without reuse, consuming memory and adding lookup overhead for each call. For example, a function that processes streaming sensor data with ever-changing timestamps will store a new entry per call, eventually causing cache thrashing and increased GC pressure. In such cases, the decorator's dictionary look-ups and memory usage outweigh any benefit, and a simple direct call is faster.

6. AsyncIO is the better fit because the workload is I/O-bound and each request spends most of its time waiting for network responses. Creating 10,000 OS threads would exhaust system resources (stack memory, scheduler overhead). With AsyncIO, a single thread runs an event loop that schedules all coroutines, keeping memory usage low. High-level code:

```
import asyncio, aiohttp

async def fetch(session, url):
    async with session.get(url) as resp:
        return await resp.text()

async def main(urls):
    async with aiohttp.ClientSession() as sess:
        tasks = [fetch(sess, u) for u in urls]
        results = await asyncio.gather(*tasks)
        return results

urls = ['http://service/api'] * 10000
print(asyncio.run(main(urls)))
# The event loop multiplexes the 10,000 coroutines, achieving high concurrency with minimal threads.
```

Reference textbooks

Fluent Python - Luciano Ramalho

Effective Python - Brett Slatkin

Primary documentation

https://docs.python.org/3/tutorial/

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

2. Data Structures & Algorithms

Prerequisites: Python (Advanced)

Data structures and algorithms form the backbone of efficient software. They let a programmer turn a naive solution into one that scales to millions of inputs, respects memory limits, and meets real-time constraints. Understanding when to use a list versus a hash table, how a graph traversal differs from a dynamic-programming recurrence, or why a trie speeds up prefix searches is essential for any developer who wants to write robust, performant code. This chapter introduces the fundamental ideas, builds intuition through exact integer examples, and equips beginners with the mental models needed to tackle more advanced topics such as amortized analysis, balanced trees, and algorithmic design patterns.

Foundations

A data structure is a way of organizing data so that operations like insertion, deletion, lookup, and traversal can be performed efficiently. The choice of structure determines the time complexity of these operations, typically expressed with Big-O notation. Algorithms are step-by-step procedures that manipulate these structures to solve a problem. The two concepts are inseparable: an algorithm's performance is measured by how well it exploits the underlying structure. For example, searching an unsorted array requires O(n) time, while searching a sorted array with binary search drops to O(log n). Similarly, a graph can be represented with adjacency lists (compact for sparse graphs) or adjacency matrices (constant-time edge checks for dense graphs). Understanding the trade-offs of space versus time, mutability versus immutability, and worst-case versus average-case behavior is the first step toward writing code that scales. The chapter proceeds by defining each named subtopic, illustrating it with exact integer matrices or small graphs, and then showing how a Python implementation follows directly from the mathematical description.

Graphs & Trees

A graph is a set of vertices connected by edges; it can be directed or undirected, weighted or unweighted. Trees are a special case: connected, acyclic graphs with a designated root. Traversal algorithms such as depth-first search (DFS) and breadth-first search (BFS) explore vertices in systematic orders, enabling tasks like connectivity testing, shortest-path discovery in unweighted graphs, and topological sorting. The choice between adjacency list and adjacency matrix representation influences both memory usage and operation speed. For sparse graphs, an adjacency list (list of neighbor lists) uses O(V+E) space, while an adjacency matrix always consumes O(V^2) space but allows O(1) edge existence checks. Tree-specific operations like inorder, preorder, and postorder traversals are recursive by nature and enable efficient search in binary search trees (BSTs).

Worked reasoning: Consider an undirected graph with vertices {1,2,3,4} and edges {(1,2),(2,3),(3,4)}. Its adjacency list is: 1:[2], 2:[1,3], 3:[2,4], 4:[3]. A BFS starting at 1 visits vertices in order 1,2,3,4. If we store the same graph as a 4x4 adjacency matrix M, M[1][2]=M[2][1]=1, M[2][3]=M[3][2]=1, M[3][4]=M[4][3]=1, all other entries 0. Checking whether an edge exists between 1 and 3 requires O(1) time with M but O(deg(1))=1 with the list. For a tree, take a binary tree with root 5, left child 3, right child 8. An inorder traversal yields 3,5,8, which is useful for producing sorted output from a BST.

Dynamic Programming

Dynamic programming (DP) solves problems by breaking them into overlapping subproblems, storing intermediate results to avoid recomputation. The two classic DP styles are top-down memoization (recursive with a cache) and bottom-up tabulation (iterative filling of a table). DP is most effective when the problem exhibits optimal substructure: the optimal solution can be composed from optimal solutions of sub-instances. Typical examples include the Fibonacci sequence, knapsack, and longest common subsequence. The key design steps are: define the state variables, write the recurrence relation, decide the iteration order, and choose a storage scheme (full table, rolling array, or hash map). Space can often be reduced from O(n^2) to O(n) by keeping only the previous row or column when the recurrence depends on a limited window.

Worked reasoning: Compute the number of ways to climb a staircase of 5 steps, taking 1 or 2 steps at a time. Let f(i) be ways to reach step i. Recurrence: f(i)=f(i-1)+f(i-2) with base f(0)=1, f(1)=1. Tabulation: i:0 1 2 3 4 5; f:1 1 2 3 5 8. Thus there are 8 distinct sequences, e.g., 1-1-1-1-1, 2-2-1, etc. A memoized recursive Python function would store computed f(i) in a dict, avoiding exponential blow-up.

Trie & Hash Indexes

A trie (prefix tree) stores strings character by character in a tree where each edge represents a symbol. It enables O(k) lookup, insertion, and prefix queries, where k is the length of the key, independent of the number of stored keys. Tries are memory-heavy but excel when many keys share prefixes, such as dictionary word lists or autocomplete engines. Hash indexes, on the other hand, map keys to buckets using a hash function, providing average O(1) lookup and insertion. Collisions are resolved by chaining (linked lists) or open addressing (probing). The trade-off is between deterministic worst-case time (tries guarantee O(k)) and average-case constant time (hashes) with lower memory overhead. Choosing between them depends on key distribution, required operations (prefix search vs exact match), and memory constraints.

Worked reasoning: Insert the words {"cat","car","dog"} into a trie. The root has children 'c' and 'd'. Under 'c' we have 'a' leading to two branches: 't' (end of "cat") and 'r' (end of "car"). Under 'd' we have 'o'->'g' (end of "dog"). Searching for prefix "ca" traverses root->'c'->'a' and finds two completions. A hash table with simple modulo hash on the string length might map "cat"->bucket 3, "car"->bucket 3 (collision) and store both in a list; lookup requires scanning that list, O(1) average but O(n) worst case.

Worked examples

Worked example

```
def bfs(adj, start):
    from collections import deque
    visited=set([start])
    q=deque([start])
    order=[]
    while q:
        v=q.popleft()
        order.append(v)
        for nb in adj[v]:
            if nb not in visited:
                visited.add(nb); q.append(nb)
    return order

adj={1:[2],2:[1,3],3:[2,4],4:[3]}
print(bfs(adj,1))
```

The function implements breadth-first search using a queue. Starting from vertex 1, it visits vertices level by level, producing the order [1, 2, 3, 4] for the linear chain graph. The algorithm runs in O(V+E) time because each vertex is enqueued once and each edge examined once. Limitations: it assumes the graph is stored as an adjacency list of Python lists; for very large graphs a generator-based neighbor iterator would reduce memory pressure. Also, BFS does not guarantee shortest weighted paths; for weighted graphs Dijkstra's algorithm is required.

Applied example

```
def knapsack(weights, values, capacity):
    n=len(weights)
    dp=[[0]*(capacity+1) for _ in range(n+1)]
    for i in range(1,n+1):
        w=weights[i-1]; v=values[i-1]
        for c in range(capacity+1):
            if w<=c:
                dp[i][c]=max(dp[i-1][c], dp[i-1][c-w]+v)
            else:
                dp[i][c]=dp[i-1][c]
    return dp[n][capacity]

print(knapsack([1,3,4],[15,20,30],5))
```

This bottom-up DP solves the 0/1 knapsack problem. The table dp[i][c] stores the maximum value achievable with the first i items and capacity c. For the sample data, the optimal selection is items 1 (weight 1, value 15) and 2 (weight 3, value 20) giving total value 35, which the function returns. The algorithm runs in O(n*capacity) time and uses O(n*capacity) space; a rolling array could cut space to O(capacity). The example uses small integer inputs so the result can be verified by hand, avoiding floating-point rounding issues.

Process flow

1. 1. Identify the problem type (search, optimization, prefix query, etc.).
2. 2. Choose a data structure that offers the best asymptotic operation for the core operation.
3. 3. Model the algorithmic steps as recurrence or traversal, noting base cases.
4. 4. Write a concrete implementation in Python, using exact integer examples for testing.
5. 5. Verify correctness by hand-computing small instances and comparing outputs.
6. 6. Analyze time and space complexity using Big-O, checking worst-case scenarios.
7. 7. Optimize memory (rolling arrays, in-place updates) or replace structure if trade-offs demand.

Interview and exam checkpoints

Explain the difference between an adjacency list and an adjacency matrix. When would you prefer each?

An adjacency list stores for each vertex a list of its neighbors, using O(V+E) space; edge lookup is O(deg(v)). An adjacency matrix is a V×V boolean or weight matrix, using O(V^2) space; edge existence checks are O(1). Use a list for sparse graphs where E << V^2 to save memory, and a matrix for dense graphs or when constant-time edge queries are critical, such as in Floyd-Warshall.

How does memoization turn exponential recursion into polynomial time?

Memoization caches the result of each distinct subproblem the first time it is computed. Subsequent calls return the cached value in O(1) instead of recomputing. For a problem like Fibonacci, the naive recursion makes 2^n calls, but memoization ensures each of the n distinct arguments is evaluated once, yielding O(n) time and O(n) space.

What are the trade-offs between a trie and a hash table for storing strings?

A trie provides deterministic O(k) lookup and natural support for prefix queries, at the cost of higher memory because each character creates a node. A hash table offers average O(1) exact-match lookup with lower memory overhead, but cannot efficiently answer prefix queries and suffers from worst-case O(n) lookup if many collisions occur. Choose a trie when prefix search or ordered traversal is required; choose a hash table for simple key-value maps with many distinct keys.

Describe how you would detect a cycle in a directed graph.

Perform a depth-first search while maintaining a recursion stack (or color marking). When visiting a node, mark it as 'visiting'. If during DFS you encounter a neighbor already marked 'visiting', a back edge exists and a cycle is present. After all descendants are processed, mark the node as 'visited'. This runs in O(V+E) time and O(V) space.

Why is the bottom-up DP for knapsack O(n*capacity) and not exponential?

The DP builds a table where each cell dp[i][c] represents the optimal value for the subproblem of the first i items and capacity c. There are exactly n*capacity distinct subproblems, each solved in constant time using previously computed cells. No subproblem is recomputed, eliminating the exponential blow-up of the naive recursive formulation.

Give an example where using a binary search tree degrades to O(n) performance.

If keys are inserted in sorted order into an unbalanced binary search tree, the tree becomes a linked list. Each insertion and lookup then traverses all previous nodes, yielding O(n) time per operation instead of the expected O(log n) for a balanced tree.

Summary

Data structures provide the scaffolding for algorithms; the right scaffold determines whether an operation runs in linear, logarithmic, or constant time. Graphs and trees illustrate traversal patterns and space representations, while dynamic programming shows how overlapping subproblems can be solved efficiently by caching. Tries and hash indexes exemplify the trade-off between deterministic prefix performance and average-case constant lookup. By following a systematic flow - problem identification, structure selection, recurrence formulation, implementation, verification, and complexity analysis - students can build reliable, scalable solutions. Mastery of these foundations prepares learners for more advanced topics such as balanced search trees, amortized analysis, and graph algorithms with weighted edges.

Practice questions

1. Given the undirected graph with adjacency list {1:[2,3],2:[1,4],3:[1,4],4:[2,3]}, run BFS from vertex 1 and list the visitation order.

2. Compute the number of distinct ways to reach step 6 using jumps of 1, 2, or 3 steps.

3. Insert the strings "apple", "app", "ape", "bat" into a trie and show the node count.

4. Write Python code to compute the longest common subsequence length of "ABCD" and "ACBD" using DP.

5. Design a data structure to support insert(key), delete(key), and findMin() in O(log n) time. Explain your choice and trade-offs.

6. Given weights [2,5,3] and values [40,100,60] with capacity 5, solve the 0/1 knapsack by hand and verify with code.

Practice solutions

1. Initialize queue with [1]; visited={1}. Dequeue 1, add neighbors 2 and 3 to queue => queue=[2,3]; visited={1,2,3}. Dequeue 2, add neighbor 4 (not visited) => queue=[3,4]; visited={1,2,3,4}. Dequeue 3, its neighbors 1 and 4 are already visited. Dequeue 4, neighbors 2 and 3 visited. Order = [1,2,3,4].

2. Let f(i)=f(i-1)+f(i-2)+f(i-3) with base f(0)=1, f(1)=1, f(2)=2. Table: i:0 1 2 3 4 5 6; f:1 1 2 4 7 13 24. Thus 24 ways.

3. Root has children 'a' and 'b' (2 nodes). Under 'a': child 'p' (1). Under 'ap': child 'p' (1) and 'e' (1). Under 'app': child 'l' (1). Under 'appl': child 'e' (1). Under 'b': child 'a' (1). Under 'ba': child 't' (1). Total nodes = 2+1+1+1+1+1+1 = 8 nodes (excluding root).

4.
```
def lcs(a,b):
    m,n=len(a),len(b)
    dp=[[0]*(n+1) for _ in range(m+1)]
    for i in range(1,m+1):
        for j in range(1,n+1):
            if a[i-1]==b[j-1]:
                dp[i][j]=dp[i-1][j-1]+1
            else:
                dp[i][j]=max(dp[i-1][j],dp[i][j-1])
    return dp[m][n]
```

```
print(lcs('ABCD','ACBD'))
# Output 3 ("ABD" or "ACD")
```

5. A balanced binary search tree (e.g., AVL or Red-Black tree) satisfies all three operations in O(log n). Insert and delete maintain height balance via rotations, guaranteeing logarithmic depth. findMin() follows leftmost pointers from the root, also O(log n). Trade-offs: extra memory for parent pointers and balance info; worst-case constant factors higher than a hash table for exact lookup, but provides ordered iteration and range queries which hashes cannot.

6. Enumerate subsets: {} value 0 weight 0; {2} value 40 w2; {5} value 100 w5; {3} value 60 w3; {2,5} weight7>5 invalid; {2,3} weight5 value100; {5,3} weight8 invalid; {2,5,3} weight10 invalid. Best value = 100 (either item weight5 alone or items 2+3). Python code confirms: knapsack([2,5,3],[40,100,60],5) returns 100.

Reference textbooks

Introduction to Algorithms - Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein

Algorithm Design - Jon Kleinberg and Eva Tardos

Primary documentation

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

3. Docker & Containerization

Prerequisites: Python (Advanced)

Docker has become the de facto tool for packaging applications into lightweight, portable units called containers. By isolating the runtime environment, Docker lets developers run the same code on a laptop, a test server, or a cloud instance without worrying about missing libraries or mismatched system settings. Containerization shines when you need reproducible builds, rapid scaling, or clean separation between services. For data-science and deep-learning workloads, the ability to embed GPU drivers and CUDA libraries inside a container makes it possible to move heavy compute jobs across machines while preserving performance. This chapter introduces the core ideas of Docker and containerization, then builds up to the three advanced topics required for the competency: Multi-Stage GPU builds, CUDA base containers, and Docker Compose. By the end, a beginner will be able to write a Dockerfile that produces a minimal GPU-ready image, understand how to layer CUDA dependencies, and orchestrate multiple services with Compose.

Foundations

Docker works by creating a layered filesystem where each instruction in a Dockerfile adds a new read-only layer on top of the previous one. At runtime, Docker combines these layers with a thin writable layer, forming a complete root filesystem for the container. The isolation is provided by Linux kernel features: namespaces (process, network, mount, etc.) give each container its own view of the system, while cgroups limit CPU, memory, and I/O usage. Images are identified by a name and tag (e.g., myapp:1.0) and stored in a registry such as Docker Hub. When you run a container, Docker creates a new process tree whose PID 1 lives inside the container's PID namespace, ensuring that signals and process tables are scoped. For GPU workloads, the NVIDIA Container Toolkit adds a device plugin that exposes the host's GPU devices inside the container, and it also mounts the necessary driver files. Understanding these building blocks - layers, namespaces, cgroups, and the role of the toolkit - allows you to reason about why a multi-stage build can shrink an image, how a CUDA base image supplies the correct driver version, and how Docker Compose can launch a web service together with a GPU-accelerated worker without manual networking steps.

Multi-Stage GPU Builds

A multi-stage build splits the Dockerfile into separate stages, each with its own base image. The first stage typically uses a heavyweight image that contains compilers, build tools, and the full CUDA toolkit needed to compile GPU code. After the binary is built, a second stage starts from a minimal runtime image (often nvidia/cuda:xx-runtime) and copies only the compiled binaries and required libraries. This approach reduces the final image size dramatically, which speeds up distribution and reduces attack surface. The trade-off is a slightly more complex Dockerfile and the need to ensure that the runtime stage includes all shared libraries the binary depends on; missing a library will cause runtime failures.

Worked reasoning: Consider a simple CUDA program that adds two vectors. In stage one we use nvidia/cuda:12.2-devel as the base, copy the source, and run nvcc to produce a binary called vecadd. In stage two we switch to nvidia/cuda:12.2-runtime, copy the vecadd binary, and set the entrypoint. The final image contains only the runtime libraries and the binary, resulting in an image under 200 MB compared to the 2 GB size of the full development image.

CUDA Base Containers

CUDA base containers are official images published by NVIDIA that bundle a specific version of the CUDA driver, runtime, and optional cuDNN or TensorRT libraries. They are versioned by the CUDA toolkit release (e.g., 12.2) and by the OS distribution (e.g., ubuntu22.04). Using a CUDA base image guarantees binary compatibility between the container's CUDA libraries and the host's GPU driver, because the NVIDIA Container Toolkit checks that the host driver is at least as new as the container's runtime. The main trade-off is that you must align your host driver version with the container's CUDA version; otherwise the container will refuse to start. Selecting a runtime-only image reduces size, while a devel image includes compilers for on-the-fly builds.

Worked reasoning: A data-science team wants to run PyTorch with CUDA 12.2. They start FROM nvidia/cuda:12.2-runtime-ubuntu22.04, then pip install torch==2.2.0+cu122. The container inherits the correct libcuda.so and cudnn libraries, and the host's driver (>=525) satisfies the runtime check. If the host had an older driver, the container would emit an error like "CUDA driver version is insufficient for CUDA runtime version".

Docker Compose

Docker Compose is a declarative YAML format that defines multiple services, their images, networking, volumes, and environment variables. It allows you to spin up an entire application stack with a single command (docker-compose up). For containerized GPU workloads, Compose can specify the runtime: nvidia for each service that needs GPU access. Compose also handles service dependencies, so a web API can wait for a GPU-enabled worker to be ready before accepting requests. The downside is that Compose adds another layer of configuration; errors in the YAML (such as mismatched indentation) can cause the whole stack to fail to start, and debugging requires understanding both Compose and the underlying Docker commands.

Worked reasoning: A simple Compose file defines two services: 'api' built from ./api using a Python base image, and 'worker' built from ./worker using a multi-stage GPU build. Both services declare runtime: nvidia and expose ports 8000 and 8080 respectively. When docker-compose up is run, Docker creates a private network, attaches both containers, and mounts a shared volume for model files, enabling the API to send inference jobs to the worker seamlessly.

Worked examples

Worked example

```
FROM nvidia/cuda:12.2-devel-ubuntu22.04 AS build
WORKDIR /app
COPY vecadd.cu .
RUN nvcc -o vecadd vecadd.cu

FROM nvidia/cuda:12.2-runtime-ubuntu22.04
COPY --from=build /app/vecadd /usr/local/bin/vecadd
ENTRYPOINT ["vecadd"]
```

The Dockerfile above demonstrates a two-stage GPU build. The first stage uses the devel image to compile vecadd.cu, producing an executable that links against the CUDA driver. The second stage starts from a lightweight runtime image and copies only the executable. When you build this Dockerfile (docker build -t vecadd:latest .) the resulting image is roughly 180 MB, far smaller than the 2.1 GB devel image.

Running the container (docker run --gpus all vecadd:latest) executes the vector addition on the host GPU. The limitation is that any additional shared libraries required by vecadd must be present in the runtime image; otherwise the container will fail with "error while loading shared libraries".

Applied example

```
services:
  api:
    build: ./api
    ports:
      - "8000:8000"
    runtime: nvidia
    environment:
      - MODEL_PATH=/models/resnet.pt
    volumes:
      - model_data:/models
  worker:
    build: ./worker
    runtime: nvidia
    depends_on:
      - api
    volumes:
      - model_data:/models
volumes:
  model_data:
```

This Docker Compose snippet defines two services that share a named volume called model_data. The 'api' service runs a Flask server exposing port 8000, while the 'worker' service runs a GPU-enabled inference loop. Both services request the NVIDIA runtime, ensuring they have access to the host's GPUs. The depends_on clause guarantees that Docker starts the API before the worker, but it does not enforce health checks; for production you would add a healthcheck to verify the API is ready before the worker begins processing. This composition lets developers develop and test the full stack locally with a single command, and the same file can be used in CI pipelines or on a cloud host with GPU nodes.

Process flow

1. Write a simple CUDA program and verify it runs on the host GPU.
2. Create a Dockerfile using a CUDA devel base, copy the source, and compile with nvcc.
3. Add a second stage that switches to a CUDA runtime base and copies the compiled binary.
4. Build the multi-stage image and test it with docker run --gpus all to ensure GPU access.
5. Wrap the image in a Docker Compose service, adding any additional services needed.
6. Use docker-compose up to launch the full stack and verify inter-service communication.
7. Iterate by adding environment variables, volumes, or healthchecks to improve robustness.

Interview and exam checkpoints

What is the purpose of a multi-stage Docker build for GPU applications?

A multi-stage build separates the compilation environment from the runtime environment. The first stage uses a heavy CUDA devel image to compile GPU code, while the second stage uses a minimal CUDA runtime image to run the compiled binary. This reduces the final image size, lowers attack surface, and speeds up deployment, but requires careful copying of all dependent libraries.

How does the NVIDIA Container Toolkit enable GPU access inside a container?

The toolkit installs a runtime hook that, when the container is started with --gpus or runtime: nvidia, mounts the host's /dev/nvidia* devices and the driver files into the container's namespace. It also sets LD_LIBRARY_PATH so that the container sees the host's libcuda.so, ensuring binary compatibility between the container's CUDA runtime and the host driver.

Explain the difference between a CUDA base image tagged 'runtime' and one tagged 'devel'.

A 'runtime' image contains only the CUDA driver libraries needed to run pre-compiled binaries; it is small and suitable for production. A 'devel' image includes the full CUDA toolkit, compilers, and headers, allowing you to compile code inside the container. Using 'devel' for building and 'runtime' for execution yields smaller, more secure images.

What are common pitfalls when using Docker Compose with GPU services?

Common issues include forgetting to set runtime: nvidia for each GPU-using service, leading to "no CUDA devices found" errors; mismatched driver versions between host and container causing runtime version errors; and neglecting healthchecks, so dependent services start before the GPU service is ready, causing request failures.

Summary

Docker abstracts the operating system into layered images, providing isolation via namespaces and resource limits via cgroups. For GPU workloads, the NVIDIA Container Toolkit bridges host GPUs into containers, while CUDA base images supply the correct driver libraries. Multi-stage builds let you compile with a heavyweight devel image and ship a lean runtime image, dramatically shrinking size and improving security. Docker Compose orchestrates multiple containers, handling networking, volumes, and service dependencies, and can be configured to request GPU resources for any service. Mastering these concepts enables you to create reproducible, portable, and efficient GPU-accelerated applications that run consistently from a developer laptop to a cloud GPU node.

Practice questions

1. Write a Dockerfile that builds a CUDA program from source and produces a runtime-only image. Use CUDA 11.8 and a simple hello.cu that prints "Hello GPU".

2. Given a host with NVIDIA driver version 525, which of the following base images will run: nvidia/cuda:12.1-runtime, nvidia/cuda:12.2-runtime, nvidia/cuda:11.8-runtime? Explain.

3. In a Compose file, you have a service 'web' that depends on a GPU-enabled service 'worker'. How would you ensure 'web' only starts after 'worker' reports healthy?

4. What trade-offs would you consider when choosing between a single-stage Dockerfile that uses nvidia/cuda:12.2-devel and a multi-stage build that ends with nvidia/cuda:12.2-runtime?

5. Implement a Python script that runs inside a CUDA runtime container and prints the number of visible GPUs using PyTorch. Provide the code and expected output.

6. Design a containerized pipeline for training a model on GPU and serving it via a REST API. List the services, their base images, and how they communicate.

Practice solutions

1. FROM nvidia/cuda:11.8-devel-ubuntu20.04 AS build
WORKDIR /src
COPY hello.cu .
RUN nvcc -o hello hello.cu

FROM nvidia/cuda:11.8-runtime-ubuntu20.04
COPY --from=build /src/hello /usr/local/bin/hello
ENTRYPOINT ["hello"]

Build with: docker build -t hello-gpu .
Run with: docker run --gpus all hello-gpu
The program will output "Hello GPU" from the device.

2. The host driver 525 supports CUDA runtime up to 12.1. Therefore nvidia/cuda:12.1-runtime will run, nvidia/cuda:12.2-runtime will fail with a version mismatch error, and nvidia/cuda:11.8-runtime will also run because older runtimes are compatible with newer drivers.

3. Add a healthcheck to the 'worker' service that runs a small command like 'nvidia-smi' and returns 0 when successful. Then set depends_on with condition: service_healthy. Example:

```
worker:
  build: ./worker
  runtime: nvidia
  healthcheck:
    test: ["CMD", "nvidia-smi"]
    interval: 5s
    retries: 3
web:
  build: ./web
  depends_on:
    worker:
      condition: service_healthy
```

4. A single-stage image is simpler to write and guarantees all build-time dependencies are present at runtime, reducing the risk of missing libraries. However, it is large (often >2 GB) and includes compilers

that are unnecessary in production, increasing attack surface. A multi-stage build produces a much smaller image (~200 MB), faster to pull and start, and is more secure, but requires careful identification of runtime dependencies and adds complexity to the Dockerfile.

5. import torch
print('GPUs visible:', torch.cuda.device_count())
Expected output on a machine with 2 GPUs: GPUs visible: 2
Run with: docker run --gpus all my-pytorch-image python script.py

6. Services:
1. trainer: FROM nvidia/cuda:12.2-devel-ubuntu22.04, installs PyTorch, runs training script, writes model to a shared volume /models.
2. api: FROM python:3.11-slim, installs Flask and torch, mounts /models, loads the trained model on startup, serves /predict endpoint.
Communication: Both services share a named Docker volume 'model_data'. The trainer writes model.pt to /models; the API reads it. No network calls needed between them. Use Docker Compose to start both; trainer runs once, then exits, while api stays up serving requests.

Reference textbooks

Docker Deep Dive - Nigel Poulton

Docker: Up & Running - Sean P. Kane and Karl Matthias

Primary documentation

https://docs.docker.com/get-started/

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

4. Advanced SQL & Data Modeling

Prerequisites: Python (Advanced)

Advanced SQL and data modeling are essential for turning raw data into actionable insight in modern applications. While basic SELECT statements retrieve rows, advanced techniques let you compute running totals, rank results, traverse hierarchical data, and understand how the database engine turns a query into a plan. Mastering these tools enables you to write queries that are both expressive and performant, a skill highly prized in data-intensive roles. This chapter introduces the core concepts of window functions, recursive common table expressions, and query execution plans, building a solid foundation before you tackle optimization and complex modeling tasks.

Foundations

SQL is a declarative language: you describe *what* result you need, not *how* to compute it. The database optimizer interprets the statement, rewrites it, and chooses an execution strategy based on statistics, indexes, and cost models. Understanding this process is crucial because advanced features such as window functions and recursive CTEs rely on the optimizer to allocate resources efficiently. A window function computes a value across a set of rows related to the current row, without collapsing the result set, using clauses like PARTITION BY and ORDER BY. Recursive CTEs allow you to define a query that references its own output, enabling traversal of trees or graphs. Query execution plans are visual or textual representations of the steps the engine will take, showing scans, joins, sorts, and memory usage. By reading a plan you can spot missing indexes, expensive sorts, or unnecessary row expansions, and then adjust your SQL or schema accordingly. These foundations give you the mental model to predict performance and to write correct, maintainable queries.

Window Functions

Window functions extend aggregate capabilities by operating over a sliding frame of rows defined relative to the current row. Unlike GROUP BY, they preserve the original row count, allowing calculations such as running totals, moving averages, and rank without collapsing data. The OVER clause specifies PARTITION BY to group rows and ORDER BY to define the frame order. Frame clauses like ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW control the exact subset. Trade-offs include increased memory usage for large partitions and potential sorting overhead; however, they often replace complex self-joins, making queries easier to read and maintain.

Worked reasoning: Consider a sales table with columns (region, sale_date, amount). To compute a cumulative sales total per region ordered by date, use: SELECT region, sale_date, amount, SUM(amount) OVER (PARTITION BY region ORDER BY sale_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cum_total FROM sales; This returns each row with its running sum, preserving all original rows while avoiding a self-join.

Recursive CTEs

Recursive common table expressions let a query refer to its own output, enabling hierarchical traversals such as organizational charts or bill-of-materials explosions. A recursive CTE consists of an anchor member that provides the base rows and a recursive member that joins the CTE to other tables, typically using a UNION ALL. The recursion stops when the join yields no new rows, preventing infinite loops. Trade-offs involve careful termination conditions and potential exponential row growth; indexing the join columns and limiting recursion depth are common safeguards.

Worked reasoning: Given an employee table (emp_id, manager_id, name), retrieve the full management chain for employee 5: WITH RECURSIVE chain AS (SELECT emp_id, manager_id, name, 0 AS level FROM employee WHERE emp_id = 5 UNION ALL SELECT e.emp_id, e.manager_id, e.name, c.level+1 FROM employee e JOIN chain c ON e.emp_id = c.manager_id) SELECT * FROM chain ORDER BY level; This yields each manager up to the CEO, with a level indicating distance from the original employee.

Query Execution Plans

An execution plan is the database's roadmap for fulfilling a query. It lists operators such as Table Scan, Index Seek, Hash Join, and Sort, along with estimated row counts, costs, and memory usage. The plan is generated by the optimizer, which chooses the lowest-cost path based on statistics. Reading a plan helps you identify bottlenecks: a full table scan on a large table suggests a missing index; a costly Sort may be avoided with pre-sorted indexes; a high-cost Hash Join could be replaced by a Merge Join if inputs are already ordered. Trade-offs involve balancing index maintenance overhead against query speed, and sometimes accepting a higher CPU cost to reduce I/O.

Worked reasoning: For SELECT * FROM orders WHERE customer_id=42, inspect estimated versus actual rows in EXPLAIN ANALYZE. If one customer owns 50 of a million rows, a suitable index on customer_id can avoid reading the whole table. If the predicate returns most rows, a sequential scan may be the better plan. Compare execution time and buffer reads before and after the index; do not interpret optimizer cost units as milliseconds. Refresh stale statistics before forcing a different plan.

Worked examples

Worked example

```
import sqlite3
conn = sqlite3.connect(':memory:')
c = conn.cursor()
c.execute('CREATE TABLE sales(region TEXT, sale_date TEXT, amount INT)')
c.executemany('INSERT INTO sales VALUES (?,?,?)', [
    ('East','2023-01-01',100),
    ('East','2023-01-02',150),
    ('West','2023-01-01',200),
    ('East','2023-01-03',120)
])
query = '''
SELECT region, sale_date, amount,
       SUM(amount) OVER (PARTITION BY region ORDER BY sale_date
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cum
FROM sales ORDER BY region, sale_date;
'''
for row in c.execute(query):
    print(row)
conn.close()
```

The script creates a tiny sales table, inserts four rows, and runs a window function that computes a cumulative total per region. The output shows each original row plus a new column 'cum' with the running sum: ('East','2023-01-01',100,100), ('East','2023-01-02',150,250), ('East','2023-01-03',120,370), ('West','2023-01-01',200,200). Because the window is partitioned by region, the West row starts its own total at 200. This example demonstrates that no self-join is needed, but note that SQLite must materialize the partition, which can increase memory usage for large partitions.

Applied example

```
WITH RECURSIVE nums(n) AS (
  SELECT 1
  UNION ALL
  SELECT n+1 FROM nums WHERE n < 5
)
SELECT n, n*n AS square FROM nums;
```

The recursive CTE generates numbers 1 through 5 without a physical numbers table. The anchor SELECT 1 starts the recursion; the recursive member adds 1 until the condition n < 5 fails. The final SELECT computes each number's square. This pattern is useful for generating date ranges, pagination offsets, or test data. In production, ensure a LIMIT or MAXRECURSION setting to avoid runaway loops, and consider indexing the generated column if it will be joined later.

Process flow

1. 1. Identify the business question and decide whether aggregation, ranking, or hierarchy is needed.
2. 2. Choose a window function if you need row-wise calculations that preserve the result set.
3. 3. If the data is hierarchical, design a recursive CTE with a clear anchor and termination condition.
4. 4. Write the SQL query, adding PARTITION BY, ORDER BY, and frame clauses as required.
5. 5. Execute EXPLAIN (or EXPLAIN ANALYZE) to view the execution plan and spot scans or sorts.
6. 6. Add or adjust indexes based on the plan, then re-run the query to verify cost reduction.
7. 7. Validate results against a small hand-calculated dataset to ensure correctness.

Interview and exam checkpoints

Explain how a window function differs from a GROUP BY aggregation.

A window function computes a value across a set of rows related to the current row while keeping each input row in the output. GROUP BY collapses rows into a single result per group, losing the original row granularity. Window functions use the OVER clause with optional PARTITION BY and ORDER BY to define the frame, allowing calculations like running totals, ranks, or moving averages without a separate join. This makes them more expressive for analytical queries, though they may require extra sorting or memory for large partitions.

What are the termination conditions for a recursive CTE and why are they important?

A recursive CTE stops when the recursive member produces no new rows, which is the natural termination condition. Additionally, many databases enforce a maximum recursion depth (e.g., 100) to prevent infinite loops. Proper termination is crucial because without it the query could run indefinitely, consuming CPU and memory. Designing the anchor and recursive parts to converge - such as moving up a manager hierarchy until a NULL manager_id is reached - ensures predictable execution and protects the system.

How can you use an execution plan to decide whether to add an index?

When you run EXPLAIN, look for operators like TABLE SCAN or SORT that have high estimated costs. If a query filters on a column but the plan shows a full scan, adding an index on that column can turn the scan into an INDEX SEEK, reducing I/O. Similarly, if a Sort appears before a join, a composite index matching the ORDER BY can eliminate the sort. However, you must weigh the write overhead of maintaining the index against the read performance gains.

Describe a scenario where a window function could be replaced by a self-join, and discuss trade-offs.

To compute a running total, you could self-join the table on date <= current date and sum the amounts. This works but typically requires a GROUP BY on the joined rows, leading to O(n^2) row combinations and higher memory usage. A window function achieves the same result with a single pass and internal sorting, offering better performance and clearer syntax. The trade-off is that some older databases lack window support, forcing the self-join approach.

Summary

Advanced SQL equips you to answer analytical questions efficiently. Window functions let you compute aggregates across a moving frame while preserving rows, ideal for rankings and cumulative metrics. Recursive CTEs provide a declarative way to walk hierarchical data without procedural code, but require careful termination logic. Execution plans reveal how the optimizer will execute your query, guiding index creation and highlighting costly operations. By mastering these tools, you can write concise, high-performance queries, diagnose bottlenecks, and design schemas that support complex analytical workloads.

Practice questions

1. Given a table sales(product_id INT, sale_date TEXT, qty INT) with rows (1,'2023-01-01',10), (1,'2023-01-02',15), (2,'2023-01-01',5), compute the cumulative quantity per product using a window function.

2. Write a recursive CTE to list all ancestors of employee 4 given employee(emp_id, manager_id) rows (1,NULL),(2,1),(3,1),(4,2). Show the output.

3. A query plan shows a full table scan on orders (1,000,000 rows) followed by a sort on order_date. Which index would most likely eliminate both operations?

4. Explain why a window function with ROWS BETWEEN 2 PRECEDING AND CURRENT ROW might produce different results than using AVG(amount) GROUP BY with a HAVING clause.

5. Implement in Python a function that executes the recursive CTE from the previous practice question and returns the ancestor list as a Python list.

6. Design trade-off: You need to support both fast point lookups by order_id and frequent queries that compute a running total of order amounts per customer. Which indexing strategy would you choose and why?

Practice solutions

1. SQL: SELECT product_id, sale_date, qty,
SUM(qty) OVER (PARTITION BY product_id ORDER BY sale_date
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cum_qty
FROM sales ORDER BY product_id, sale_date;
Result rows: (1,'2023-01-01',10,10), (1,'2023-01-02',15,25), (2,'2023-01-01',5,5).

2. WITH RECURSIVE anc AS (
SELECT emp_id, manager_id FROM employee WHERE emp_id = 4
UNION ALL
SELECT e.emp_id, e.manager_id FROM employee e JOIN anc a ON e.emp_id = a.manager_id
)
SELECT emp_id FROM anc WHERE emp_id <> 4;
Output: 2, 1 (ancestors ordered from direct manager upward).

3. Create a composite index on orders(order_date). The index can satisfy the ORDER BY, allowing the optimizer to use an INDEX SEEK that returns rows already sorted, removing both the table scan and the explicit sort.

4. The window frame limits the calculation to the current row and the two rows before it, producing a moving average that changes for each row. GROUP BY aggregates over the entire partition, yielding a single average per group. HAVING filters groups after aggregation, not per row, so the semantics differ entirely.

5. import sqlite3

```
def get_ancestors(emp_id):
    conn = sqlite3.connect(':memory:')
    cur = conn.cursor()
    cur.execute('CREATE TABLE employee(emp_id INT, manager_id INT)')
    cur.executemany('INSERT INTO employee VALUES (?,?)', [(1,None),(2,1),(3,1),(4,2)])
    query = '''
    WITH RECURSIVE anc AS (
      SELECT emp_id, manager_id FROM employee WHERE emp_id = ?
      UNION ALL
      SELECT e.emp_id, e.manager_id FROM employee e JOIN anc a ON e.emp_id = a.manager_id
    )
    SELECT emp_id FROM anc WHERE emp_id <> ?;
    '''
    cur.execute(query, (emp_id, emp_id))
    result = [row[0] for row in cur.fetchall()]
    conn.close()
```

```
    return result
# get_ancestors(4) returns [2, 1]
```

6. Create a primary key index on order_id for O(log n) point lookups. Additionally, add a composite index on (customer_id, order_date, amount). This index supports the ORDER BY needed for the window function, allowing the database to read rows already sorted by date per customer, eliminating a separate sort. The trade-off is extra storage and write overhead for maintaining the composite index, but it yields fast point lookups and efficient running-total calculations.

Reference textbooks

Learning SQL - Alan Beaulieu

SQL Performance Explained - Markus Winand

Primary documentation

https://www.postgresql.org/docs/current/tutorial.html

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

5. REST API & Service Design

Prerequisites: Python (Advanced)

REST (Representational State Transfer) APIs have become the lingua franca for web services because they are simple, stateless, and built on ubiquitous HTTP primitives. Understanding how to design a REST API that is robust, evolvable, and secure is essential for any backend engineer. This chapter explains the core principles of RESTful service design, walks through concrete examples in Python, and shows how idempotency, versioning, data validation with Pydantic, and authentication middleware fit together. By mastering these foundations, a developer can build services that scale, are easy to test, and survive changes without breaking existing clients.

Foundations

A REST API is defined by resources, each identified by a stable URI, and a uniform interface that uses standard HTTP methods (GET, POST, PUT, PATCH, DELETE). The stateless constraint means every request must contain all information needed to process it; the server never stores client context between calls. Representations of resources are typically JSON, but any media type is allowed. Content negotiation lets clients request the format they prefer via the Accept header. Caching is controlled with Cache-Control and ETag headers, enabling intermediaries to reduce load. Errors follow the HTTP status code taxonomy, with 4xx for client errors and 5xx for server faults. Designing an API begins with modeling the domain as resources, choosing appropriate URIs, and mapping CRUD operations to HTTP verbs while respecting idempotency: GET, PUT, DELETE are idempotent, whereas POST is not. Versioning protects clients from breaking changes; common strategies include URI versioning (/v1/users) and header versioning (Accept: application/vnd.myapi.v2+json). Validation of request payloads prevents malformed data from reaching business logic; in Python, Pydantic provides declarative models that parse and validate JSON, raising clear errors. Finally, authentication middleware intercepts each request, extracts credentials (e.g., JWT tokens), verifies them, and injects the user identity into the request context, ensuring that downstream handlers can enforce authorization rules.

Idempotency & Versioning

Idempotency guarantees that repeating the same request produces the same effect on the server and the same response, which is crucial for reliability in distributed systems where retries happen automatically. GET, PUT, and DELETE are defined as idempotent; a client can safely retry them without fearing duplicate side effects. POST is not idempotent because it typically creates a new resource each time. To make a POST operation idempotent, a client can supply an idempotency key (a unique token) that the server stores and checks before creating a resource. Versioning isolates breaking changes. URI versioning is explicit and easy to route, while header versioning keeps URLs clean but requires more sophisticated content negotiation. Both approaches must be documented so clients know which contract they are using. Trade-offs include URL clutter versus header complexity, and the need to maintain multiple versions in code.

Worked reasoning: Consider a payment service with endpoint POST /payments that creates a transaction. A client may lose the response due to a network glitch and retry the request. If the server generates a new transaction ID each time, the client could be charged twice. By requiring an Idempotency-Key header, the server stores the key with the transaction ID on the first request. On a retry with the same key, the server returns the original transaction ID, ensuring only one charge. For versioning, the original API uses

/v1/payments returning fields amount, currency. In v2 the service adds a field status. Clients that still call /v1/payments receive the older schema, while new clients use /v2/payments, avoiding breaking existing integrations.

Pydantic Validation

Pydantic is a Python library that uses type hints to define data models and automatically validates incoming JSON against those models. Each field can have a type, default value, and constraints such as regex patterns or numeric ranges. Validation occurs at model instantiation; if the input does not conform, Pydantic raises a ValidationError containing a detailed error map. This eliminates boilerplate parsing code and centralizes schema definitions. Pydantic also supports custom validators for complex logic, and it can export models to JSON schemas for API documentation. The trade-off is a slight runtime overhead for validation, but the safety and clarity it provides outweigh the cost in most web services.

Worked reasoning: A user registration endpoint expects a JSON payload with username, email, and age. Using Pydantic, we define:

```
class UserIn(BaseModel):
    username: constr(min_length=3, max_length=30)
    email: EmailStr
    age: conint(ge=13)
```

If a client sends {"username":"ab","email":"not-an-email","age":10}, Pydantic raises a ValidationError listing three violations: username too short, email invalid, age below minimum. The API can catch this exception and return a 422 Unprocessable Entity response with a structured error body, guiding the client to correct the request.

Auth Middleware

Authentication middleware sits early in the request processing pipeline. Its job is to extract credentials (often a Bearer token in the Authorization header), verify them (e.g., by checking a JWT signature or consulting an OAuth introspection endpoint), and attach the resulting user identity to the request context. This decouples authentication from business logic, allowing route handlers to assume a verified user object is present. Middleware must also handle token expiration, revocation, and scope checking. A common pattern is to raise an HTTPException with status 401 if authentication fails, or 403 if the user lacks required permissions. The trade-off is added latency for each request due to token verification, which can be mitigated with caching of public keys or token introspection results.

Worked reasoning: In a FastAPI application, an auth middleware might look like:

```
@app.middleware("http")
async def auth_middleware(request: Request, call_next):
    token = request.headers.get("Authorization", "").replace("Bearer ", "")
    try:
        payload = jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"])
        request.state.user = payload["sub"]
    except jwt.ExpiredSignatureError:
        return JSONResponse(status_code=401, content={"detail":"Token expired"})
    except jwt.InvalidTokenError:
        return JSONResponse(status_code=401, content={"detail":"Invalid token"})
```

```
    response = await call_next(request)
    return response
```

All downstream endpoints can now read request.state.user without re-validating the token.

Worked examples

Worked example

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, EmailStr, conint
import uuid
app = FastAPI()

class PaymentIn(BaseModel):
    amount: conint(gt=0)
    currency: str
    idempotency_key: str

payments = {}

@app.post("/v1/payments")
def create_payment(p: PaymentIn):
    if p.idempotency_key in payments:
        return payments[p.idempotency_key]
    pid = str(uuid.uuid4())
    record = {"id": pid, "amount": p.amount, "currency": p.currency}
    payments[p.idempotency_key] = record
    return record
```

The code defines a FastAPI service with a POST /v1/payments endpoint. Pydantic validates that amount is a positive integer and that an idempotency_key is supplied. The in-memory dictionary payments stores results keyed by the idempotency token. On the first request, a new UUID is generated and stored; on a retry with the same key, the previously stored record is returned, demonstrating idempotent behavior. Limitations include the volatile in-memory store (lost on restart) and lack of authentication; in production you would replace the dict with a persistent database and add auth middleware to protect the endpoint.

Applied example

```
def versioned_route(path, version):
    return f"/v{version}{path}"

# Example usage
user_url = versioned_route("/users", 2)
# user_url == "/v2/users"

# Pseudocode for auth check
if request.headers.get('Authorization') is None:
    reject(401)
else:
    user = verify_jwt(request.headers['Authorization'])
    if not user.has_scope('payments:write'):
        reject(403)
    else:
        process()
```

The first function builds a versioned URI by concatenating the version number with the base path, illustrating a simple URI versioning scheme. The second block is pseudocode for an authentication check that would live in middleware. It first ensures an Authorization header exists, then verifies the JWT and checks that the token includes the required scope. If any step fails, the request is rejected with the appropriate HTTP status. This pattern separates concerns: routing logic stays clean while security is enforced uniformly across all endpoints.

Process flow

1. 1. Model the domain as resources and assign stable URIs.
2. 2. Choose HTTP methods for each operation, ensuring idempotent methods where retries are expected.
3. 3. Define request and response schemas using Pydantic models, adding field constraints.
4. 4. Implement authentication middleware that validates tokens and populates request context.
5. 5. Add idempotency handling for non-idempotent POST endpoints by storing idempotency keys.
6. 6. Version the API using URI prefixes or Accept headers, and document each version's contract.
7. 7. Write automated tests that cover validation errors, auth failures, and idempotent retries.

Interview and exam checkpoints

Explain why GET is considered idempotent and how that influences client retry logic.

GET is idempotent because invoking it multiple times does not change server state; each call returns the same representation of the resource (assuming no external changes). This property allows clients and intermediaries to safely retry GET requests after transient failures without risking duplicate side effects, which is essential for robust HTTP communication.

How would you design a versioning strategy for a public API that must support both backward compatibility and rapid iteration?

I would adopt URI versioning (e.g., /v1/, /v2/) for major breaking changes, because it makes the version explicit in routing and documentation. For minor, non-breaking additions, I would use content negotiation with custom media types (Accept: application/vnd.myapi.v1+json) so clients can opt-in to new fields. This hybrid approach preserves backward compatibility while allowing rapid iteration on optional features.

What are the advantages of using Pydantic over manual JSON parsing in a FastAPI endpoint?

Pydantic automatically validates types, enforces constraints, and produces clear error messages, reducing boilerplate code. It also generates JSON Schema for OpenAPI documentation, ensuring the API contract is always in sync with the code. Manual parsing would require repetitive checks, is error-prone, and lacks automatic documentation generation.

Describe how authentication middleware can be made stateless and why that matters.

Stateless authentication middleware validates a token (e.g., JWT) that contains all necessary claims and is signed by the issuer. Because the token itself carries the user identity and expiration, the middleware does not need to query a session store, keeping each request independent. Statelessness improves scalability, as any server instance can handle any request without shared session state.

Summary

Designing a REST API involves modeling resources, applying the uniform HTTP interface, and respecting constraints such as statelessness and idempotency. Versioning protects clients from breaking changes, while idempotency keys make unsafe POST operations safe to retry. Pydantic brings declarative validation, reducing bugs and generating documentation automatically. Authentication middleware centralizes security, enabling stateless token verification and consistent user context. By following a disciplined flow - from resource modeling through testing - developers can build services that are reliable, evolvable, and easy to consume.

Practice questions

1. A client sends a POST /v1/orders with payload {"amount": -5, "currency": "USD"}. Using the Pydantic model OrderIn(amount: conint(gt=0), currency: str), what response should the API return?

2. Implement a simple idempotent POST endpoint for creating a user profile using an in-memory dict and an Idempotency-Key header. Show the code and explain how duplicate requests are handled.

3. Design a trade-off discussion: you must choose between URI versioning and header versioning for a high-traffic public API. List pros and cons and state which you would pick.

4. Given a JWT token that expires in 5 minutes, what should the auth middleware return if the token is expired, and why?

5. Write a Python function that validates a list of integers is strictly increasing using Pydantic's validator decorator. Show a short test case.

6. A GET request to /v1/items returns a JSON array of 100 objects. The client wants only items where "type" == "book" without changing the server. Propose a solution using query parameters and explain how the server should handle it.

Practice solutions

1. Pydantic will raise a ValidationError because amount is not greater than 0. The API should catch this exception and return HTTP 422 with a body like {"detail":[{"loc":["body","amount"],"msg":"ensure this value is greater than 0","type":"value_error.number.not_gt"}]}. This informs the client that the amount field is invalid.

2. from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel, EmailStr
app = FastAPI()

```
class ProfileIn(BaseModel):
    name: str
    email: EmailStr

store = {}

@app.post("/v1/profiles")
def create_profile(profile: ProfileIn, idempotency_key: str = Header(...)):
    if idempotency_key in store:
        return store[idempotency_key]
    pid = len(store) + 1
    record = {"id": pid, "name": profile.name, "email": profile.email}
    store[idempotency_key] = record
    return record
```

Explanation: The endpoint extracts the Idempotency-Key header. If the key already exists in the store, the previously created record is returned, preventing duplicate profiles. Otherwise a new profile is created, stored under the key, and returned. This guarantees that retries with the same key are safe.

3. URI versioning pros: simple routing, clear to clients, easy caching per version. Cons: URL clutter, requires redeployment of routing tables for each version. Header versioning pros: clean URLs, can negotiate multiple versions via Accept, easier to deprecate older versions without changing routes. Cons: adds complexity to content negotiation, many HTTP clients need custom header handling, caching proxies may ignore custom Accept values. For a high-traffic public API where simplicity and cache friendliness are paramount, I would pick URI versioning because it works with CDNs and browsers out of the box, reducing operational risk.

4. The middleware should return HTTP 401 Unauthorized with a response body indicating "Token expired". A 401 signals that authentication failed and that the client may retry after obtaining a fresh token. Using 403 would be inappropriate because the credentials were valid but no longer active.

5. from pydantic import BaseModel, validator

```
class StrictList(BaseModel):
    values: list[int]

    @validator('values')
    def check_strict(cls, v):
        if any(v[i] >= v[i+1] for i in range(len(v)-1)):
            raise ValueError('list must be strictly increasing')
        return v
```

```
# Test case
try:
    StrictList(values=[1, 3, 5])
    print('valid')
except Exception as e:
    print(e)

try:
    StrictList(values=[1, 2, 2])
except Exception as e:
    print(e)

# Output: valid \n 1 validation error for StrictList \n values \n list must be strictly increasing
(type=value_error)
```

6. The client can request /v1/items?type=book. The server parses the optional "type" query parameter and filters the collection before serialization. Implementation example in FastAPI:

```
@app.get("/v1/items")
def list_items(type: str | None = None):
    all_items = get_all_items()
    if type:
        all_items = [i for i in all_items if i["type"] == type]
    return all_items
```

This keeps the endpoint generic while allowing clients to retrieve subsets, reducing bandwidth and processing on the client side.

Reference textbooks

API Design Patterns - JJ Geewax

Designing Web APIs - Brenda Jin, Saurabh Sahni and Amir Shevat

Primary documentation

https://fastapi.tiangolo.com/tutorial/

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

6. Automated Testing & CI/CD

Prerequisites: Python (Advanced)

Automated testing and continuous integration/continuous deployment (CI/CD) have become the backbone of modern software delivery. They enable teams to catch defects early, guarantee that new changes do not break existing functionality, and ship value to users at a rapid, reliable pace. This chapter explains why a disciplined testing strategy combined with an automated pipeline is essential, when to introduce each component in a project lifecycle, and how the three related concepts - pytest fixtures and mocks, GitHub Actions matrix builds, and coverage gates - fit together to create a robust quality gate. By the end of the reading, a beginner will understand the fundamental ideas, be able to write deterministic tests, configure a simple CI workflow, and enforce a minimum test coverage threshold, laying the groundwork for more advanced topics such as property-based testing, canary releases, and infrastructure as code.

Foundations

Testing starts with the notion of a test case: a small, isolated piece of code that exercises a single unit of behavior and asserts an expected outcome. In Python, the pytest framework provides a lightweight syntax for declaring such cases, automatically discovering files named test_*.py and functions prefixed with test_. Fixtures are reusable objects that set up state before a test runs and tear it down afterward; they eliminate duplication and make tests deterministic. Mocks replace real dependencies with controllable stand-ins, allowing the test to focus on the unit under test without invoking external services. Continuous integration (CI) is the practice of automatically building and testing code whenever a change is pushed to a version-control repository. GitHub Actions offers a hosted CI service where a workflow file describes jobs, steps, and triggers. A matrix strategy runs the same job across multiple Python versions or operating systems, ensuring cross-environment compatibility. Finally, a coverage gate is a rule that fails the CI job if the measured test coverage falls below a predefined percentage, encouraging developers to write sufficient tests. Together, these pieces form a feedback loop: developers write code, push it, the CI pipeline runs tests in parallel across environments, measures coverage, and reports results back to the team, preventing regressions before they reach production.

Pytest Fixtures & Mocks

A fixture is a function annotated with @pytest.fixture that returns a value used by one or more test functions. Fixtures can have scopes (function, class, module, session) that control how often they are instantiated, reducing expensive setup work. Mocks, typically created with unittest.mock.patch or pytest-mock's mocker fixture, substitute real objects with objects that record calls and can be programmed to return specific values. The trade-off is between realism and speed: a fully integrated test that hits a database provides high confidence but runs slowly, while a mock-based unit test runs fast but may miss integration bugs. Proper use of fixtures and mocks yields fast, reliable tests that still reflect the contract of the code under test.

Worked reasoning: Suppose we have a function fetch_user(id) that calls an external HTTP API. A fixture creates a dummy response object, and a mock patches requests.get to return that dummy. The test then calls fetch_user(42) and asserts that the returned user name matches the mocked payload. This isolates network I/O, makes the test deterministic, and runs in milliseconds.

GitHub Actions Matrix

A matrix in a GitHub Actions workflow defines a set of variables (e.g., python-version: [3.9, 3.10, 3.11]) that the runner expands into separate jobs, each with a unique combination of those variables. This enables parallel testing across environments without duplicating workflow code. The matrix can also include operating system values (ubuntu-latest, windows-latest) to verify cross-platform behavior. The main downside is increased CI minutes consumption; careful selection of matrix dimensions balances coverage with cost. Matrix failures are reported per combination, making it easy to pinpoint environment-specific issues.

Worked reasoning: A workflow file declares a job "test" with strategy.matrix.python-version set to 3.9, 3.10, and 3.11. The steps install dependencies, run pytest, and upload coverage. When a pull request is opened, GitHub spawns three parallel runners, each executing the same test suite under a different interpreter, and reports three separate status checks.

Coverage Gates

Coverage measurement records which lines of source code were executed during a test run. Tools like coverage.py produce a report with a total percentage and per-file breakdowns. A coverage gate is a CI step that fails the job if the total coverage is below a threshold, for example 85%. This enforces a minimum testing effort and highlights untested code. However, coverage is a proxy, not a guarantee of quality; 100% coverage can still miss logical errors. Therefore, coverage gates should be combined with thoughtful test design rather than used as a sole metric.

Worked reasoning: In a GitHub Actions workflow, after running pytest --cov=app, a step runs coverage report --fail-under=85. If the current commit only achieves 78% coverage, the step exits with a non-zero status, causing the entire job to be marked as failed, and the pull request cannot be merged until coverage is improved.

Worked examples

Worked example

```
import pytest
from unittest.mock import patch

# code under test
import requests

def fetch_user(user_id):
    resp = requests.get(f'https://api.example.com/users/{user_id}')
    return resp.json()['name']

# fixture providing dummy JSON
@pytest.fixture
def dummy_response():
    class Dummy:
        def json(self):
            return {'name': 'Alice'}
    return Dummy()

def test_fetch_user(dummy_response):
    with patch('requests.get', return_value=dummy_response) as mock_get:
        name = fetch_user(42)
        assert name == 'Alice'
        mock_get.assert_called_once_with('https://api.example.com/users/42')
```

The test defines a fixture dummy_response that mimics the JSON payload returned by the real API. Using unittest.mock.patch, the test replaces requests.get with a function that returns this dummy object. When fetch_user is called, it receives the mocked response, extracts the name, and the assertion verifies the expected value. The mock also records the call arguments, allowing the test to confirm that the correct URL was constructed. This pattern isolates network I/O, makes the test fast and deterministic, and demonstrates the interplay of fixtures and mocks. Limitations include that the test does not verify the handling of error status codes or network timeouts; separate tests would be needed for those edge cases.

Applied example

```
# pseudocode for GitHub Actions workflow
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: [3.9, 3.10, 3.11]
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: ${{ matrix.python-version }}
      - name: Install deps
        run: pip install -r requirements.txt
      - name: Run tests with coverage
        run: pytest --cov=app
      - name: Enforce coverage gate
        run: coverage report --fail-under=85
```

The workflow triggers on pushes and pull requests. The job "test" runs on Ubuntu and defines a matrix over three Python versions. Each matrix entry checks out the code, installs dependencies, runs pytest with coverage collection, and finally executes coverage report with a fail-under threshold of 85%. If any matrix run produces less than 85% coverage, that specific job fails, and the overall CI status shows a red check. This example illustrates how to combine fixtures, mocks, and coverage gates in a real CI pipeline, providing immediate feedback to developers about both functional correctness and test completeness across environments.

Process flow

1. 1. Write a deterministic unit test using pytest, employing fixtures for shared setup.
2. 2. Replace external calls with mocks to keep the test fast and isolated.
3. 3. Add the test file to the repository and commit the changes.
4. 4. Create a .github/workflows/ci.yml file that defines a matrix job for required Python versions.
5. 5. In the workflow, install dependencies, run pytest with the --cov flag, and generate a coverage report.
6. 6. Add a coverage gate step that fails if total coverage is below the agreed threshold.
7. 7. Push the branch; GitHub Actions automatically executes the matrix, reports results, and blocks merging if any step fails.

Interview and exam checkpoints

What is the difference between a fixture's function scope and session scope in pytest?

Function scope creates a fresh fixture instance for each test function that uses it, ensuring complete isolation but incurring setup cost per test. Session scope creates a single instance for the entire test session, shared across all tests, which is useful for expensive resources like a database container. The trade-off is between isolation (function) and performance (session); using session scope for mutable state can cause test interference if not carefully managed.

How does a GitHub Actions matrix improve test reliability?

A matrix runs the same test suite across multiple environment combinations (e.g., Python versions, OSes) in parallel. This uncovers environment-specific bugs such as version-dependent API changes or path handling differences. By reporting each combination separately, developers can pinpoint the exact configuration that fails, leading to faster diagnosis and higher confidence that the code works universally.

Explain why 100% code coverage does not guarantee bug-free software.

Coverage measures which lines were executed, not whether the exercised paths exercised all logical conditions. A test may run a line but not trigger edge cases, exception handling, or incorrect assumptions. Moreover, coverage does not assess data quality, performance, or security aspects. Therefore, high coverage is a useful indicator of test breadth but must be complemented by thoughtful test design, property-based testing, and code reviews to achieve true reliability.

When would you prefer integration tests over unit tests with mocks?

Integration tests are preferred when the interaction between components is complex, such as database transactions, message queues, or external APIs where contract compliance matters. They validate that the system works as a whole, catching configuration or compatibility issues that unit tests with mocks cannot see. However, they are slower and more brittle, so they should complement, not replace, fast unit tests that verify individual logic in isolation.

Summary

Automated testing and CI/CD form a feedback loop that catches defects early, enforces quality standards, and accelerates delivery. Pytest provides a simple syntax for writing isolated unit tests, while fixtures and mocks manage shared state and external dependencies efficiently. GitHub Actions matrix builds ensure that code behaves consistently across multiple runtimes, and coverage gates enforce a baseline of test thoroughness. Together these tools create a disciplined workflow: write deterministic tests, mock out side effects, configure a parallel CI pipeline, and fail builds that do not meet coverage goals. By mastering these foundations, developers can scale their testing strategy, reduce manual QA effort, and maintain confidence as the codebase grows.

Practice questions

1. Write a pytest fixture that creates a temporary file with the content "hello" and yields its path. Ensure the file is deleted after the test.

2. Given a function add(a, b) that returns a+b, write a test using unittest.mock.patch to mock add and verify it is called with (2,3).

3. Create a GitHub Actions matrix that tests on Python 3.9 and 3.10 on both ubuntu-latest and windows-latest. Show the YAML snippet.

4. Your CI pipeline fails the coverage gate with 82% coverage while the threshold is 85%. Explain two ways to improve coverage without adding meaningless tests.

5. Design a trade-off discussion: you can run a full matrix (3 Python versions x 2 OS) or a reduced matrix (only latest Python on Ubuntu). What factors influence your decision?

6. Given the following code, compute the exact line coverage percentage. Assume the test suite executes only the branch where x > 0.

```
def foo(x):
    if x > 0:
        return 'pos'
    else:
```

```
        return 'non-pos'

# test calls foo(5) only.
What is the coverage?
```

Practice solutions

1.
```
import pytest, tempfile, os

@pytest.fixture
def temp_hello_file():
    fd, path = tempfile.mkstemp()
    with os.fdopen(fd, 'w') as f:
        f.write('hello')
    yield path
    os.remove(path)

# In a test you can use:
# def test_file_content(temp_hello_file):
# with open(temp_hello_file) as f:
# assert f.read() == 'hello'
```

2.
```
import pytest
from unittest.mock import patch

# module mymod.py
# def add(a, b):
# return a + b

def test_add_called():
    with patch('mymod.add') as mock_add:
        from mymod import add
        add(2, 3)
        mock_add.assert_called_once_with(2, 3)

# The test passes if add is invoked exactly once with the expected arguments.
```

3.
```
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
    python-version: [3.9, 3.10]

# This creates four jobs: (ubuntu,3.9), (ubuntu,3.10), (windows,3.9), (windows,3.10).
```

4. First, identify uncovered lines using coverage report -m; then write targeted unit tests that exercise those branches, such as error-handling paths or conditional branches not hit by existing tests. Second, refactor complex functions into smaller, pure functions that are easier to test thoroughly; this often reveals hidden logic that can be exercised with concise tests, raising coverage meaningfully.

5. Factors include the project's user base (if many customers run older Python versions, broader matrix is justified), CI cost (full matrix consumes more minutes), risk tolerance (missing OS-specific bugs vs. faster

feedback), and release cadence (high-frequency releases may favor a lean matrix). A balanced approach might run the full matrix nightly and a reduced matrix on every pull request.

6. The file has 5 executable lines: the function definition line, the if line, the return 'pos' line, the else line, and the return 'non-pos' line. The test executes the function definition, the if line, and the return 'pos' line, covering 3 of 5 lines. Coverage = (3/5)*100 = 60%.

Reference textbooks

Python Testing with pytest - Brian Okken

Continuous Delivery - Jez Humble and David Farley

Primary documentation

https://docs.pytest.org/en/stable/

Content version: learning-v1 | CareerIQ original AI-assisted educational notes